LAPORAN PRAKTIKUM

IOT 2

MODUL 4



Disusun Oleh:

Nama : Reza Aurelia Saputri

NPM : 25782080

Prodi : TRI C

Dosen:

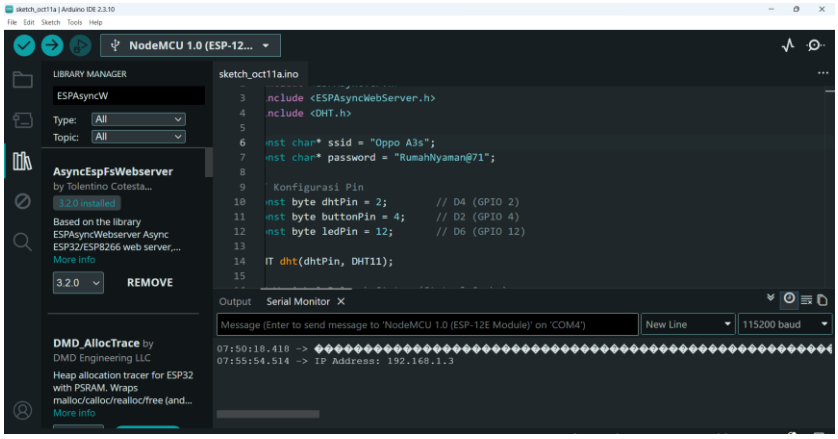
Agus Ambarwari, S.Pd., M.Kom.

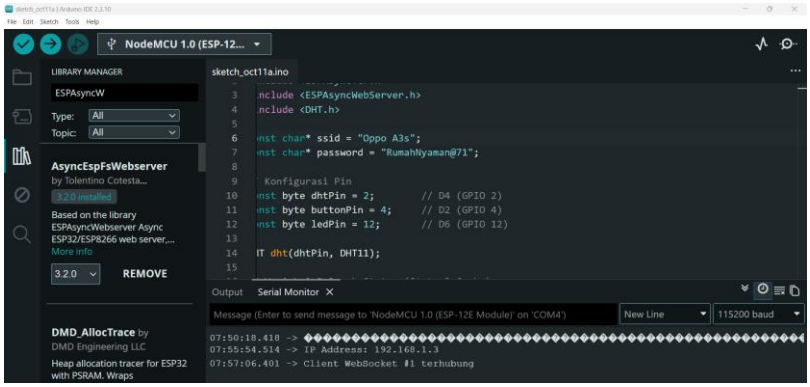
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

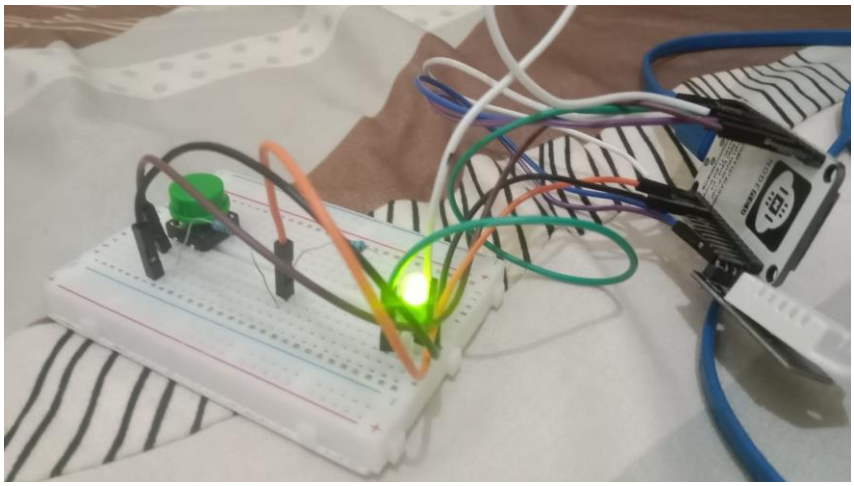
JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026


NodeMCU 1.0 (ESP-12...
LIBRARY MANAGER
ESPAsyncW
Type: All
Topic: All
AsyncEspFsWebserver
by Tolentino Cotesta...
3.2.0 installed
Based on the library ESPAsyncWebserver Async ESP32/ESP8266 web server,...
More info
3.2.0
REMOVE
DMD_AllocTrace by
DMD Engineering LLC
Heap allocation tracer for ESP32 with PSRAM. Wraps malloc/calloc/realloc/free (and...
More info
sketch_oct11a.ino
nclude <ESPAsyncWebServer.h>
nclude <DHT.h>
nst char* ssid = "Oppo A3s";
nst char* password = "RumahNyaman@71";
Konfigurasi Pin
nst byte dhtPin = 2; // D4 (GPIO 2)
nst byte buttonPin = 4; // D2 (GPIO 4)
nst byte ledPin = 12; // D6 (GPIO 12)
T dht(dhtPin, DHT11);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
07:50:18.418 ->
07:55:54.514 -> IP Address: 192.168.1.3


NodeMCU 1.0 (ESP-12...
LIBRARY MANAGER
ESPAsyncW
Type: All
Topic: All
AsyncEspFsWebserver
by Tolentino Cotesta...
3.2.0 installed
Based on the library ESPAsyncWebserver Async ESP32/ESP8266 web server,...
More info
3.2.0
REMOVE
DMD_AllocTrace by
DMD Engineering LLC
Heap allocation tracer for ESP32 with PSRAM. Wraps
sketch_oct11a.ino
nclude <ESPAsyncWebServer.h>
nclude <DHT.h>
nst char* ssid = "Oppo A3s";
nst char* password = "RumahNyaman@71";
Konfigurasi Pin
nst byte dhtPin = 2; // D4 (GPIO 2)
nst byte buttonPin = 4; // D2 (GPIO 4)
nst byte ledPin = 12; // D6 (GPIO 12)
T dht(dhtPin, DHT11);
Output Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
New Line
115200 baud
07:50:18.418 ->
07:55:54.514 -> IP Address: 192.168.1.3
07:57:06.401 -> Client WebSocket #1 terhubung



Pembahasan :

Praktikum ini menggunakan NodeMCU ESP8266, LED, *push button*, dan sensor DHT11/DHT22 untuk membuat sistem *Real-Time Dashboard*. WebSocket digunakan sebagai komunikasi dua arah antara NodeMCU dan *browser* sehingga data dapat diperbarui secara *real-time* tanpa melakukan *refresh* halaman.

LED dapat dikontrol melalui tombol pada *browser* maupun *push button* fisik. Ketika *push button* ditekan, status LED berubah dan informasi tersebut langsung dikirim ke *browser* melalui WebSocket. Sebaliknya, perintah dari *browser* juga dapat mengubah kondisi LED fisik. Sistem juga membaca suhu dari sensor DHT dan menampilkannya pada *dashboard*.

Hasil praktikum menunjukkan bahwa komunikasi antara perangkat keras dan web dapat berlangsung secara *real-time* tanpa *refresh* halaman, sehingga sistem dapat melakukan sinkronisasi dua arah antara tombol fisik, LED, dan *browser*.

Latihan 1 (Teori Jaringan)

Soal:

Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/*bandwidth* internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Jawaban:

WebSockets lebih hemat bandwidth karena menggunakan satu koneksi yang tetap terbuka antara browser dan NodeMCU. Data dapat dikirim secara langsung ketika ada pembaruan tanpa harus membuat permintaan HTTP baru berulang kali.

Sementara itu, Meta Refresh HTTP memuat ulang halaman secara berkala sehingga menghasilkan permintaan berulang dan browser harus memproses ulang halaman. Hal ini dapat meningkatkan penggunaan bandwidth dan menyebabkan layar berkedip atau tampilan kembali ke kondisi awal.

Latihan 2 (Analisis Waktu/Timer)

Soal:

Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks delay(3000) di dalam siklus fungsi void loop() pada draf arsitektur kompleks ini?

Jawaban:

Penggunaan delay(3000) menyebabkan program berhenti menjalankan instruksi berikutnya selama 3 detik. Akibatnya, pemrosesan pesan WebSockets, pembaruan status LED, dan tugas lainnya dapat tertunda.

Dampaknya adalah dashboard menjadi kurang responsif, perintah tombol mengalami keterlambatan, dan koneksi WebSockets berisiko terputus apabila keterlambatan berlangsung terlalu lama. Oleh karena itu, sebaiknya digunakan timer non-blocking dengan fungsi millis() agar program tetap dapat menjalankan tugas lain selama menunggu waktu pembaruan sensor.

Latihan 3 (Modifikasi Front-End)

Soal:

Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Jawaban:

Pada program Smart Room yang digunakan, ID elemen HTML yang menjadi target perubahan teks adalah:

1. temp, untuk menampilkan nilai suhu.
2. hum, untuk menampilkan nilai kelembapan.
3. led, untuk menampilkan status LED, yaitu ON atau OFF.

Latihan 4 (Pengembangan Telemetri Sensor – Kelembapan)

Soal:

Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:

1. **Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--";). Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer non-blocking millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke browser (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
2. **Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu ( ) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.

Tuliskan baris kode modifikasi Anda pada sisi HTML, JavaScript, dan C++!

Jawabaan:

Pada Latihan 4, sistem ditambahkan fitur untuk menampilkan kelembapan secara real-time melalui WebSocket. Nilai kelembapan dibaca menggunakan fungsi readHumidity(), kemudian dikirim dalam format JSON dengan parameter hum. Pada bagian JavaScript, data hum diterima dan ditampilkan pada elemen HTML humValue. Dengan demikian, suhu dan kelembapan dapat diperbarui secara real-time tanpa melakukan refresh halaman.

